

Relatório IARTI

Sprint B

Turma 3DG _ Grupo 39

1231267 _ Rúben Alexandre Abreu Freitas

1231031 _ Rafael Oliveira Costa

1230927 _ José Luís de Oliveira Ribeiro

1222123 _ João Pedro Oliveira de Sousa

Data: 23/11/2025

Índice

Sprint B	3
User Story 3.4.2 - (1231267 – Rúben Freitas)	3
Explicação	4
Demonstração	5
User Story 3.4.3 – (1231031 – Rafael Costa)	6
Explicação	6
User Story 3.4.4 – (1230927 – José Ribeiro)	9
Explicação	9
Código.....	10
Demonstração	12
User Story 3.4.5 – (1222123 – João Sousa)	13
Explicação	14
Demonstração	18

Sprint B

User Story 3.4.2 - (1231267 – Rúben Freitas)

As a Logistics Operator, I want to generate a daily schedule for the loading and unloading operations of vessels arriving at the port on a given day, so that delays relative to desired departure times are minimized.

Acceptance Criteria / Comments:

- The objective of the scheduling algorithm is to minimize total delay between the actual completion and desired departure times of vessels.
- Currently, the scheduling algorithm must only consider:
 - a) One vessel per dock at a time.
 - b) One crane (system) per unloading/loading operation.
 - c) One storage location for the unloading/loading operations.
 - d) Availability of physical resources (crane) and qualified staff within their operational windows.
- The scheduling computation must be executed through the Planning & Scheduling back-end module, which consumes the required data (e.g., vessel arrivals/departures, resources, staff data) from other APIs.
- The scheduling process must be initiated through a dedicated interface on the SPA. This UI must:
 - a) Allow the operator to specify the target date (day).
 - b) Display the results in a summary table (e.g., vessel, start/end time, assigned crane, staff) and, if feasible, through a timeline approach.
 - c) Provide feedback on progress and completion, including warnings about infeasibility (e.g., lack of resources or staff).
- At this stage, results do not need to be persisted anywhere— they can be recomputed on demand.

Explicação

Para a execução desta User Story optei por implementar um endpoint no backend já realizado no âmbito do projeto, aproveitando a infraestrutura de segurança e seguindo também a recomendação do professor. No backend é reunida a informação necessária para enviar um pedido HTTP para o módulo de Prolog, estas informações são os Vessel Visit Notifications que tem com chegada no dia pedido e a crane associada à doca em que os barcos vão atracar. Após o envio do pedido HTTP com um DTO que reúne as informações, o módulo de prolog formata as informações recebidas e cria os factos vessel(...).

```
process_vessels([], _, []).
process_vessels([V|Rest], CraneCapacity, MaxCranes, [_|Facts]) :-
    VesselName = V.vesselIMO,
    ETAString = V.eta,
    ETDString = V.eta,
    CargoManifests = V.get(cargoManifests, []),

    datetime_to_hour(ETAString, ETAHour),
    datetime_to_hour(ETDString, ETDHour),

    % Conta containers por tipo
    count_cargo(CargoManifests, loading, NLoading),
    count_cargo(CargoManifests, unloading, NUnloading),

    compute_loading_unloading(NLoading, NUnloading, CraneCapacity, LoadingTime, UnloadingTime),

    % Cria facto vessel(..., MaxCranes)
    assertz(vessel(VesselName, ETAHour, ETDHour, UnloadingTime, LoadingTime, MaxCranes)),

    with_output_to(user_error, format("Fato criado: vessel(~w, ~2f, ~2f, ~2f, ~2f, ~w)\n",
        [VesselName, ETAHour, ETDHour, LoadingTime, UnloadingTime, MaxCranes])),
    with_output_to(user_error, format(" Containers: Loading=~w, Unloading=~w, CraneCapacity=~w\n",
        [NLoading, NUnloading, CraneCapacity])),

    process_vessels(Rest, CraneCapacity, MaxCranes, Facts).
```

```
count_cargo([], _, 0).
count_cargo([M|Rest], TypeAtom, Count) :-
    string_lower(M.manifestType, ManifestType),
    Entries = M.get(entries, []),
    length(Entries, NumEntries),
    atom_string(TypeAtom, TypeString),
    (ManifestType = TypeString -> ThisCount = NumEntries ; ThisCount = 0),
    count_cargo(Rest, TypeAtom, OtherCount),
    Count is ThisCount + OtherCount.

% Calcula tempos de carga/descarga
compute_loading_unloading(NLoading, NUnloading, CraneCapacity, LoadingTime, UnloadingTime) :-
    (CraneCapacity =:= 0 -> EffectiveCap = 1 ; EffectiveCap = CraneCapacity),
    LoadingTime is NLoading / EffectiveCap,
    UnloadingTime is NUnloading / EffectiveCap.

datetime_to_hour(DateTimeStr, HourDecimal) :-
    sub_atom(DateTimeStr, _, 1, After, "T"),
    sub_atom(DateTimeStr, After, 0, TimePart),
    split_string(TimePart, ":", "", [HStr, MStr | _]),
    number_string(H, HStr),
    number_string(M, MStr),
    HourDecimal is H + (M / 60).
```

Nestes predicados é feita a formatação, onde passa as Datas das notificações para horas (que será usado nos factos), conta a quantidade de containers por operação (Loading e Unloading) para depois calcular o tempo de unloading e loading em função da capacidade operacional da crane usada.

Já o algoritmo usado foi o fornecido pelo professor, que cria todas as sequências e seleciona a melhor, ou seja, com o menor atraso. O algoritmo segue uma abordagem de procura extensiva, mas tem a vantagem de encontrar a melhor solução possível. O algoritmo começa pelo predicado

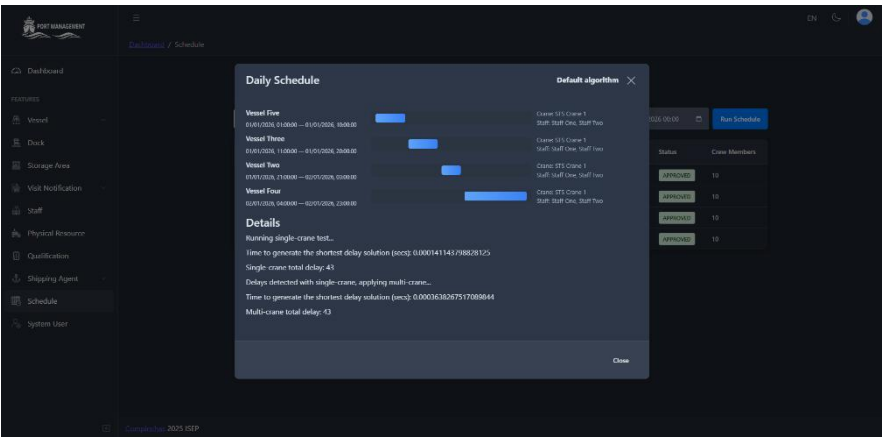
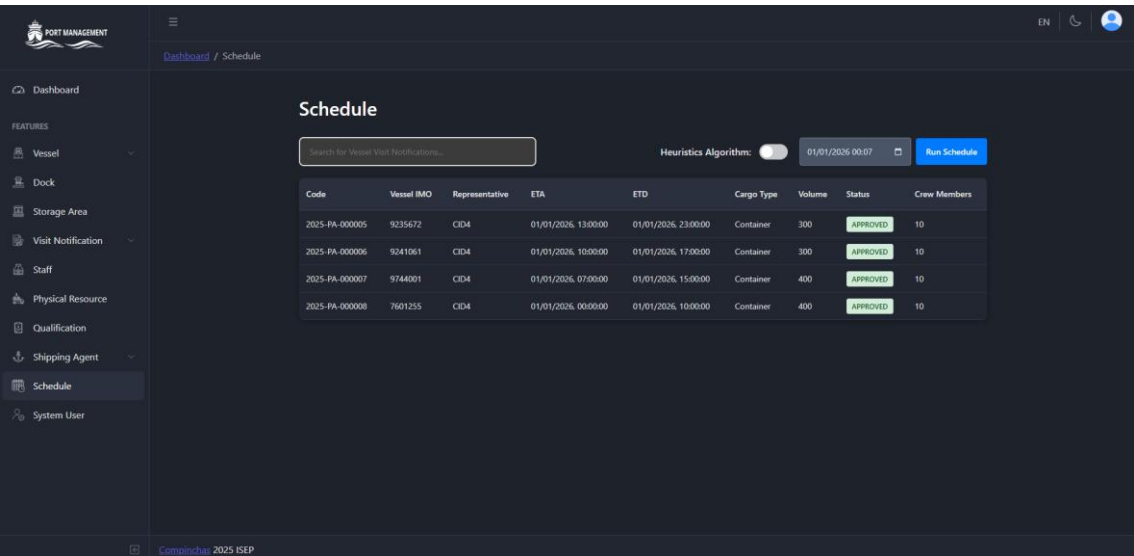
obtain_seq_shortest_delay(SeqBest, DelayBest, ExecutionTime), começa por invocar o predicado **obtain_seq_shortest_delay1**, responsável por gerar todas as permutações dos barcos com o **permutation(LV,SeqV)**, e para cada uma, calcular a respetiva agenda com o **sequence_temporization(SeqV,SeqTriplets)**. Depois é calculado o atraso total associado a cada sequência com

sum_delays(SeqTriplets,S), e o predicado final

compare_shortest_delay(SeqTriplets,S) compara os atrasos e atualiza a solução caso o atraso seja menor. No final o algoritmo retorna a melhor sequência com o menor atraso.

Demonstração

UI



User Story 3.4.3 – (1231031 – Rafael Costa)

As a Logistics Operator, I want the team to analyze the computational complexity of the scheduling algorithm, so that I can understand its scalability, feasibility and efficiency.

Acceptance Criteria / Comments:

- **The team must assess performance under different problem sizes (e.g., number of vessels, cranes, staff) when computing the optimal solution.**
- **The algorithm's complexity class must be explained and justified.**
- **The findings must be summarized in a short technical report to support further optimization or heuristic development.**

Explicação

1. Avaliação de Performance sob diferentes dimensões do problema

Foram realizados testes de performance para ambos os algoritmos de escalonamento desenvolvidos, variando o número de vessels entre 5 e 13. Os resultados demonstram comportamentos computacionais drasticamente diferentes entre as duas abordagens.

1.1 Algoritmo da user story 3.4.2

Este algoritmo, que visa minimizar o atraso total das operações através de uma solução exata, apresenta os seguintes tempos de execução:

- **5 vessels:** 0.70 ms
- **6 vessels:** 3.59 ms
- **7 vessels:** 29.64 ms
- **8 vessels:** 285.05 ms
- **9 vessels:** 2.94 segundos
- **10 vessels:** 33.12 segundos
- **11 vessels:** 6 minutos e 28 segundos
- **12 vessels:** ~76 minutos (projeção)
- **13 vessels:** ~24 horas (projeção)

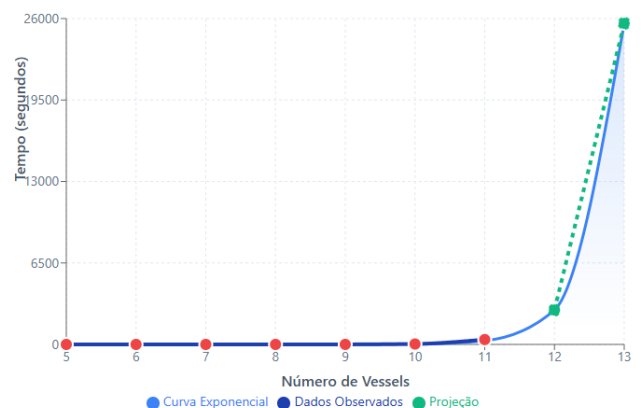


Figura 1 - Relação do número de vessels com o tempo (algoritmo da 3.4.2)

Este algoritmo apresenta uma equação exponencial:

$$F(x) = 6.257e^{-9} * e^{(2.2339*x)}$$

$x = \text{número de vessels}$

$F(x) = \text{Tempo em segundos}$

1.2 Algoritmo da user story 3.4.4

O algoritmo alternativo, que prioriza eficiência computacional através de estratégias heurísticas, apresenta tempos de execução substancialmente inferiores:

- **5 vessels:** 0.30 ms
- **6 vessels:** 0.55 ms
- **7 vessels:** 0.71 ms
- **8 vessels:** 2.27 ms
- **9 vessels:** 3.02 ms
- **10 vessels:** 4.76 ms
- **11 vessels:** 6.19 ms
- **12 vessels:** 7.95 ms
- **13 vessels:** 10.77 ms
- **14 vessels:** ~13.5 ms (projeção)
- **15 vessels:** ~17.5 ms (projeção)

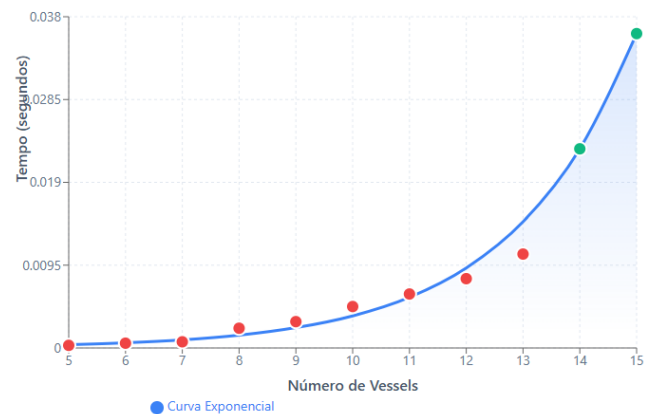


Figura 2 - Relação do número de vessels com o tempo (algoritmo da 3.4.4)

Este algoritmo apresenta uma equação exponencial:

$$F(x) = 3.832e^{-5} * e^{(0.4565*x)}$$

$x = \text{número de vessels}$

$F(x) = \text{Tempo em segundos}$

2. Análise Comparativa

2.1 Escalabilidade

Nº de Vessels	Algoritmo 3.4.2	Algoritmo 3.4.4	Rácio entre tempos
5	0.70 ms	0.30 ms	2.3×
8	285 ms	2.27 ms	125×
10	33.12 s	4.76 ms	~7,000×
11	6m 28s	6.19 ms	~63,000×
13	~24 horas	10.77 ms	~8,000,000×

O algoritmo heurístico demonstra **escalabilidade exponencialmente superior**, tornando-se até **8 milhões de vezes mais rápido** para 13 vessels.

2.2 Viabilidade Operacional

Algoritmo da user story 3.4.2:

- Viável até 8 vessels (< 1 segundo);
- Aceitável para 9 vessels (≈ 3 segundos);
- Inviável para 10+ vessels (> 30 segundos);

Algoritmo da user story 3.4.4:

- Viável até 15 vessels (< 20 ms);
- Adequado para planeamento em tempo real;

3. Conclusão

A análise de complexidade computacional revela diferenças fundamentais entre os dois algoritmos de escalonamento.

O **algoritmo da user story 3.4.2** apresenta crescimento exponencial acentuado, tornando-se inviável para mais de 10 vessels, onde o tempo de execução excede 30 segundos. Para 13 vessels, o tempo projetado é de aproximadamente 24 horas.

O **algoritmo da user story 3.4.4** demonstra crescimento exponencial significativamente mais controlado, mantendo-se eficiente até 15+ vessels com tempos inferiores a 20 milissegundos. Para 13 vessels, este algoritmo é aproximadamente **8 milhões de vezes mais rápido** que o algoritmo da user story 3.4.2.

User Story 3.4.4 – (1230927 – José Ribeiro)

As a Logistics Operator, I want an alternative scheduling algorithm for the loading and unloading operations of vessels arriving at the port on a given day, that produces a good (but not necessarily optimal) solution efficiently, so that the system can handle larger problem instances or time-constrained planning scenarios.

Acceptance Criteria / Comments:

- **This algorithm must be available for selection on the dedicated interface of the SPA and reuse the same data inputs and interfaces defined for the existing scheduling module.**
- **This algorithm must aim to minimize vessel departure delays but prioritize computational efficiency over optimality. Suitable approaches may include greedy strategies, local search, or other informed heuristics.**
- **Results must be comparable (e.g., total delay, computation time) against the previous algorithm using summary metrics.**
- **At this stage, results do not need to be persisted anywhere— they can be recomputed on demand.**

Explicação

O algoritmo visa fornecer um agendamento eficiente para as operações de carga e descarga de navios em um porto. Ao contrário do método exaustivo anterior, esta versão utiliza heurísticas informadas para gerar soluções boas rapidamente, adequadas para cenários de planejamento em tempo limitado ou com muitas embarcações.

O algoritmo é compatível com os dados existentes do módulo de agendamento, usando os mesmos inputs e interfaces.

Heurísticas utilizadas:

- **Greedy Insertion:** A construção incremental permite escolher, a cada passo, o navio que adiciona menos atraso à sequência atual. Isso produz rapidamente uma sequência sem precisar avaliar todas as permutações possíveis. Isto fará com que a complexidade computacional seja reduzida.

- **Local Improvement/Hill-Climbing:** Após a implementação da greedy insertion, a sequência pode não ser ótima. Esta heurística faz pequenas trocas de posição entre vessels podendo reduzir o delay. Permite melhorar a solução, aproveitando a sequência razoavelmente boa, melhorando o resultado sem ter um grande impacto no tempo de execução.

Esta estratégia adaptada permite alcançar uma solução com uma combinação de eficiência e qualidade, em que primeiramente construímos uma solução boa através do Greedy Insertion, e de seguida, aplica-se um refinamento para reduzir os atrasos restantes com o Local Improvement/Hill-Climbing.

Código

Greedy Insertion

Método que chama a heurística **Greedy Insertion**.

```
% Greedy insertion order
greedy_order_by_insertion(LV, OrderedLV) :-
    greedy_insertion_build(LV, [], OrderedLV).
```

- LV: lista de navios ainda não sequenciados
- OrderedLV: lista final de navios ordenado segundo a heurística
- A função chama greedy_insertion_build/3 com:
 - Rem = navios restantes (LV)
 - Acc = acumulador da sequência já contruída

```
greedy_insertion_build([], Acc, Acc).
```

Quando não há mais navios restantes (Rem = []), a sequência acumulada Acc é a sequência final.

```
greedy_insertion_build(Rem, Acc, Ordered) :-
    (Acc = [] -> CurrD = 0; (sequence_temporization_improved(Acc, AccTrip), sum_delays_improved(AccTrip, CurrD))),
    findall(Inc-V, (
        member(V, Rem),
        append(Acc, [V], NewSeqV),
        sequence_temporization_improved(NewSeqV, Trip),
        sum_delays_improved(Trip, D),
        Inc is D - CurrD
    ), Pairs),
    keysort(Pairs, Sorted),
    Sorted = [ _-BestV | _ ],
    select(BestV, Rem, Rem2),
    append(Acc, [BestV], Acc2),
    greedy_insertion_build(Rem2, Acc2, Ordered).
```

- Na primeira linha do método calculamos o atraso total atual da sequência já construída (Acc). Caso Acc estar vazio, significa que não há delay, então o currD = 0. Caso contrário sequence_temporization_improved(Acc, AccTrip) converte a sequência Acc com horários de início/fim, depois é feito sum_delays_improved(AccTrip, CurrD) em que calcula o atraso total da sequência atual (CurrD).

- No findall tem como objetivo avaliar o efeito de adicionar cada navio restante ao final da sequência atual, pois não basta adicionar o navio que chega primeiro ou tem o menor tempo de descarga, mas sim o que vai melhorar o atraso total. Para cada navio em V em Rem, append(Acc, [V], NewSeqV) cria a sequência hipotética com V adicionado no final, calculando depois os horários de início/fim e o atraso total dessa nova sequência. Inc is D - CurrD determina quanto o atraso aumenta ao adicionar V, resultando na lista Pairs.
- keysort(Pairs, Sorted) ordena os navios pelo incremento de atraso em ordem crescente, sendo que o Sorted = [_-BestV | _] escolhe o navio que menos aumenta o atraso (BestV).
- select(BestV, Rem, Rem2) → remove BestV da lista de navios restantes. append(Acc, [BestV], Acc2) → adiciona BestV à sequência acumulada. Após é feita uma chamada recursiva, em que continua a construir a sequência com os navios restantes (Rem2) e o novo acumulador (Acc2).

Local Improvement/Hill-Climbing

Após ser feita a heurística Greedy Insertion, é chamado local_improvement/3 para refinar a sequência.

```
local_improvement(SeqV, BestSeqV, BestDelay) :-
    sequence_temporization_improved(SeqV, Trip),
    sum_delays_improved(Trip, Delay),
    local_swap_optimize(SeqV, Delay, BestSeqV, BestDelay).
```

Recebe uma sequência de inicial de navios (SeqV), convertendo a sequência em triplets de horários (Trip) com sequence_temporization_improved/2, calculando o atraso total da sequência com sum_delays_improved/2, após isso chama local_swap_optimize/4 para tentar reduzir o atraso total por trocas sucessivas.

```
local_swap_optimize(Seq, Delay, BestSeq, BestDelay) :-
    length(Seq, N),
    findall(D2-Swapped, (
        between(1, N, I), Jstart is I+1, between(Jstart, N, J),
        swap_positions(Seq, I, J, Swapped),
        sequence_temporization_improved(Swapped, T2),
        sum_delays_improved(T2, D2)
    ), Results),
```

O objetivo gerar todas as trocas possíveis de pares de navios e avaliar o efeito no atraso total, em vai gerar todas as sequências que podem ser obtidas trocando um par de navios, verificando se o atraso total calculado é menor que o atraso atual.

- length(Seq, N) → número total de navios na sequência
- between(1, N, I) e between(Jstart, N, J) percorre todos os pares (I,J) com I < J para evitar repetir swaps ou trocar o mesmo navio consigo mesmo.
- swap_positions(Seq, I, J, Swapped) → cria uma nova sequência com os navios I e J trocados de posição.

- para cada sequência trocada, calcula os horários e o atraso total

O resultado será numa lista results de pares D2-Swapped, ou seja, o atraso total e a sequência correspondente.

```
( Results = [] -> BestSeq = Seq, BestDelay = Delay
; keysort(Results, Sorted), Sorted = [MinD-BestCandidate|_],
  (MinD < Delay -> local_swap_optimize(BestCandidate, MinD, BestSeq, BestDelay)
; BestSeq = Seq, BestDelay = Delay)
).
```

Ainda no predicado local_swap_optimize/4, caso Results estiver vazio, retorna a sequência atual como a melhor. Caso contrário faz keysort(Results, Sorted) em que ordena todas as sequências geradas pelas trocas pelo atraso total (D2), após é feito [MinD-BestCandidate|_] em que escolhe a sequência com menor atraso (BestCandidate). Se MinD < Delay faz uma chamada recursiva em local_swap_optimize/4 com a nova sequência (BestCandidate) e atraso atualizado (MinD). Se não há melhora, retorna a sequência original (Seq) e o seu atraso (Delay).

```
swap_positions(Seq, I, J, SeqOut) :-
  nth1(I, Seq, ElemI), nth1(J, Seq, ElemJ),
  set_nth1(Seq, I, ElemJ, Temp),
  set_nth1(Temp, J, ElemI, SeqOut).
```

O predicado swap_position/4 é auxiliar ao predicado anterior em que troca os elementos de uma lista nas posições I e J.

- nth1(I, Seq, ElemI) pega o elemento na posição I e nth1(J, Seq, Elem) pega o elemento na posição J.
- set_nth1/4 substitui ElemI por ElemJ e vice-versa, gerando SeqOut.

```
set_nth1([_|T], 1, Elem, [Elem|T]).
set_nth1([H|T], N, Elem, [H|R]) :- N>1, N1 is N-1, set_nth1(T, N1, Elem, R).
```

Predicado set_nth1/4 em que substitui o elemento na posição N da lista por Elm.

Demonstração

Para demonstrar a eficiência do algoritmo, vamos comparar com o algoritmo fornecido. Para testar foram utilizados 10 dos seguintes vessels:

```
vessel(va, 6, 63, 10, 16).
vessel(vb, 23, 50, 9, 7).
vessel(vc, 8, 40, 5, 12).
vessel(vd, 27, 40, 0, 8).
vessel(ve, 36, 70, 12, 0).
vessel(vf, 40, 60, 8, 6).
vessel(vg, 52, 80, 9, 10).
vessel(vi, 61, 90, 13, 8).
vessel(vj, 74, 100, 7, 7).
vessel(vk, 81, 110, 6, 8).
```

Podemos observar que o algoritmo fornecido demorou 32 segundos para fornecer uma solução com 10 vessels, onde o delay é 210.

```
?- obtain_seq_shortest_delay(SeqBetterTriplets, SShortestDelay).
Better Sequence: [(vc,8,24),(vb,25,40),(vd,41,48),(vf,49,62),(ve,63,74),(vg,75,93),(vj,94,107),(vk,108,121),(vl,122,142),(vm,143,168)]
Shortest Delay: 210
Time to generate the shortest delay solution: 32.7884840965271
SeqBetterTriplets = [(vc, 8, 24), (vb, 25, 40), (vd, 41, 48), (vf, 49, 62), (ve, 63, 74), (vg, 75, 93), (vj, 94, 107), (vk, ..., ...), (... , ...)]...
SShortestDelay = 210.
```

No algoritmo de heurísticas podemos observar, que para os 10 vessels demorou 0.005 segundos para fornecer a solução, em que o delay é também 210.

```
?- obtain_seq_shortest_delay_improved(SeqTriplets, DelayHours, ExecutionTime).
SeqTriplets = [(vc, 8, 24), (vb, 25, 40), (vd, 41, 48), (vf, 49, 62), (ve, 63, 74), (vg, 75, 93), (vj, 94, 107), (vk, ..., ...), (... , ...)]...,
DelayHours = 210,
ExecutionTime = 0.00571894645690918.
```

User Story 3.4.5 – (1222123 – João Sousa)

As a Logistics Operator, I want the scheduling module to support the use of multiple cranes when a single-crane solution cannot eliminate vessel departure delays, so that total delay is minimized while using additional cranes only when strictly necessary.

Acceptance Criteria / Comments:

- The system must first attempt to generate a schedule using a single-crane allocation strategy.
- If the computed schedule still results in non-zero departure delays, the module must (automatically or optionally) re-evaluate the plan allowing multiple cranes per vessel.
- The multi-crane scheduling approach must aim to:
 - Minimize the total sum of vessel departure delays.
 - Minimize the additional time windows where more than one crane is required (i.e., minimize multi-crane usage intensity).
- The output must clearly indicate where and when additional cranes were allocated to meet schedule objectives.
- The operator must be able to compare results between single-crane and multi-crane strategies via summary metrics (e.g., total delay, number of crane-hours used).
- At this stage, results do not need to be persisted anywhere— they can be recomputed on demand.

Explicação

O objetivo deste trabalho foi estender o módulo de agendamento de navios de forma a suportar a utilização de múltiplas gruas sempre que uma solução de apenas uma grua por navio não fosse suficiente para eliminar ou reduzir os atrasos nas partidas. Na implementação procurei cumprir as regras definidas na user story, garantindo eficiência, minimização de atrasos e utilização conservadora de recursos adicionais (gruas extra).

Começando pelo `vesse_schedule` temos o seguinte:

Fundamentação do modelo multi-cranes

O modelo assume que cada navio possui um limite máximo de gruas utilizáveis, representado pelo parâmetro `maxCranes`. A atribuição de múltiplas gruas tem como efeito reduzir o tempo total de serviço, aqui conceptualizado como a soma de operações de descarga e carga ($T_{Unload} + T_{Load}$). Este tempo é recalculado segundo uma função de divisão inteira superior, refletindo um comportamento de redução proporcional:

$$ExecTime = \lceil \frac{T_{Unload} + T_{Load}}{N} \rceil,$$

onde N é o número de gruas atribuídas ao navio. Este modelo corresponde a uma idealização linear do aumento de produtividade proporcionado pelo acréscimo de recursos.

Construção sistemática das alocações de gruas

O algoritmo testa sequencialmente diferentes quantidades globais de gruas N , com $N \in [1, M]$, onde M corresponde ao maior valor de `maxCranes` presente entre todos os navios. Para cada valor de N , é gerada uma lista de alocações:

- se o navio permite pelo menos N gruas $\rightarrow$ recebe N ;
- se o navio permite menos do que $N \rightarrow$ recebe o seu máximo (`maxCranes`).

Assim, o processo adota uma estratégia de **alocação uniforme**, garantindo que todos os navios beneficiam do mesmo número de gruas sempre que isso seja permitido pelas respetivas limitações.

Recomputação da temporização com múltiplas cranes

Com base na alocação uniformizada, o sistema recalcula os tempos de serviço e gera quadrupletos da forma:

$$(V, T_{in}, T_{end}, ExecTime, N),$$

onde *ExecTime* é ajustado ao número de guas atribuídas. A duração revista implica uma alteração direta do tempo de conclusão (*TEndLoad*), embora a ordem sequencial dos navios se mantenha invariável relativamente à encontrada no escalonamento single-crane. Esta preservação da ordem garante estabilidade e reduz o espaço combinatório da pesquisa.

Avaliação do atraso total

Após recalcular as durações de serviço, procede-se à determinação do atraso total:

$$Delay = \sum_i (\max(0, (TEndLoad_i + 1) - TDep_i),$$

onde *TDep_i* representa a janela de partida estipulada para o navio *i*. Esta métrica quantifica a penalidade associada ao não cumprimento das janelas temporais exigidas, constituindo o critério central para a comparação das soluções.

Mecanismo de pesquisa sequencial e seleção ótima

O algoritmo realiza uma pesquisa incremental, testando todos os valores de *N* entre 1 e o máximo permitido. O procedimento apresenta duas propriedades relevantes:

Paragem antecipada

A pesquisa termina imediatamente quando:

- o atraso calculado é igual a zero;
- o atraso calculado seria negativo, sendo este normalizado para zero.

Estes casos representam soluções ótimas, eliminando a necessidade de testar valores superiores de *N*, o que contribui para a eficiência computacional.

Seleção por minimização

Se não ocorrer paragem antecipada, o algoritmo compara o atraso obtido para cada valor de N , selecionando a configuração que minimiza o atraso total. Este critério garante que a solução multi-cranes nunca apresenta uma degradação face ao cenário single-crane.

Versão parametrizada da pesquisa multi-cranes

Uma versão complementar do procedimento, denominada `obtain_seq_shortest_delay_improved_multi/4`, permite especificar um limite superior para o número de gruas a considerar. Esta variante assume particular relevância em contextos onde existam:

- restrições operacionais impostas pelo terminal;
- interesse analítico em cenários específicos;
- necessidade de reduzir a complexidade computacional.

Apesar da limitação imposta, os princípios fundamentais do algoritmo mantêm-se inalterados.

Para a modificação do `improved_vessel_scheduler` foram então feitas as seguintes alterações ao mesmo:

Representação estruturada dos navios

No modelo original, cada navio era representado por uma tripla contendo o instante de início de operação e o instante de conclusão. A abordagem multi-guindaste amplia esta representação para um quadrupeto da forma:

[Navio, TInícioOperação, TFimOperação, NumGuindastes].

A inclusão do número de guindastes atribuídos a cada navio permite ajustar diretamente o tempo de processamento, uma vez que este passa a ser dividido pelo número de guindastes disponíveis.

Temporização com múltiplas cranes

A função `sequence_temporization_improved_multi/3` calcula os intervalos de operação considerando explicitamente a partilha de recursos. O tempo total de processamento de um navio (descarga + carga) é dividido pelo número de guindastes atribuídos. Para garantir consistência temporal, é imposto um tempo mínimo de operação de uma unidade, mesmo quando a divisão resultaria num valor inferior a um. Este mecanismo assegura uma modelação realista da redução de tempo proporcionada pela utilização de mais guindastes.

Cálculo de atrasos

A função `sum_delays_improved_multi/2` adapta o cálculo de atrasos para trabalhar sobre quadrupletos, mantendo a mesma lógica que compara o instante real de saída do navio com o seu instante desejado. Esta adaptação é essencial para permitir uma avaliação consistente das soluções geradas pelo sistema multi-guindaste.

Restrições de cranes por navio

A função `allowed_cranes_for_vessel/2` introduz um mecanismo de controlo das restrições físicas impostas pelos cais. Cada navio está associado a um cais com uma capacidade máxima de guindastes. Este módulo assegura que nenhuma solução gerada viole as limitações operacionais da infraestrutura portuária.

Construção greedy da sequência e atribuição de cranes

O algoritmo guloso foi generalizado para o contexto multi-guindaste através da função `greedy_order_by_insertion_multi/3`. Durante a inserção de cada navio na sequência parcial, o algoritmo avalia todas as combinações possíveis entre posição e número de guindastes atribuídos, selecionando aquela que induz o menor aumento no atraso total. Desta forma, a heurística passa a otimizar simultaneamente a ordenação dos navios e a alocação de guindastes.

Otimização local

A etapa de melhoria local, implementada pela função `local_improvement_multi/5`, expande a vizinhança de pesquisa para incluir dois tipos de modificações:

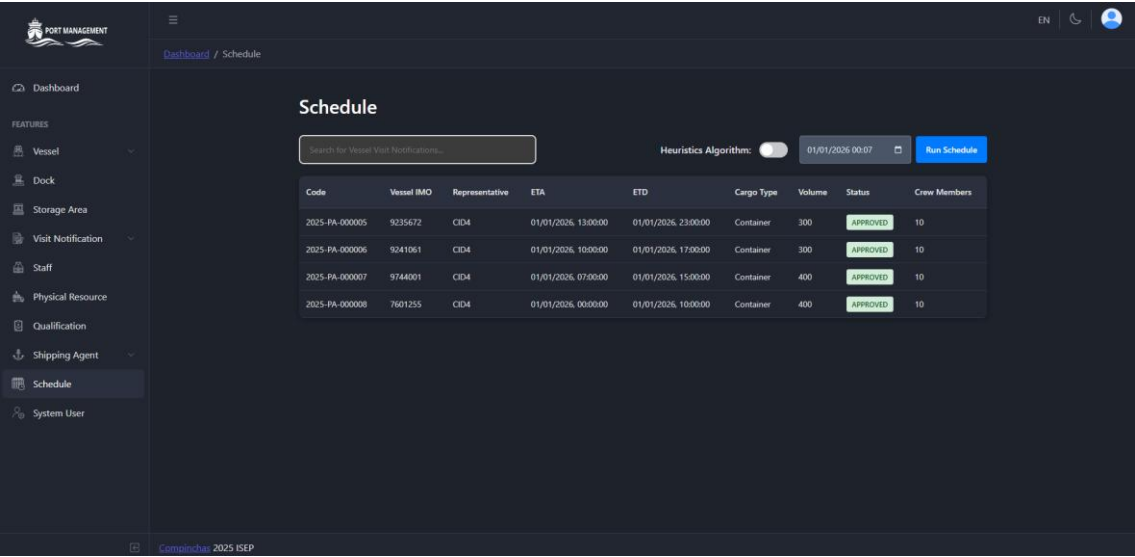
1. **Troca de posições entre navios;**
2. **Alteração do número de cranes atribuídos a cada navio,** respeitando sempre as restrições do cais.

A cada iteração, o algoritmo seleciona a melhor melhoria disponível, repetindo o processo até não existirem mais alterações que reduzam o atraso total. Esta abordagem permite escapar a soluções demasiado dependentes da heurística gulosa inicial.

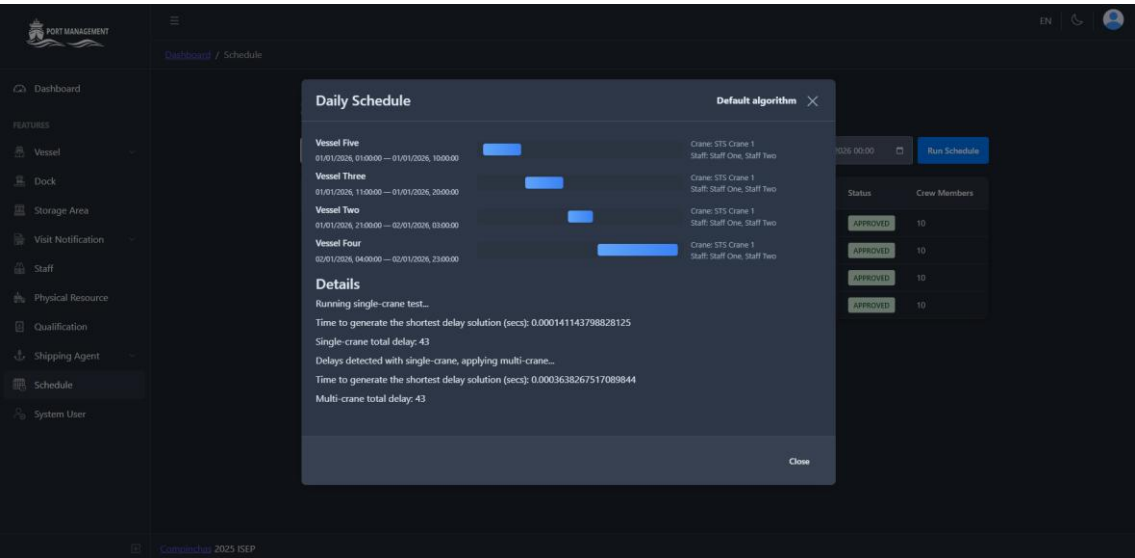
Scheduler final multi-crane

A função `schedule_greedy_improved_multi/4` integra os componentes anteriormente descritos, construindo a solução inicial, aplicando a otimização local e produzindo a sequência final de quadrupletos, juntamente com o atraso total associado. Este scheduler constitui a versão completa e funcional do sistema multi-guindaste.

Demonstração



Vessel_schedule com apenas uma crane



Vessel_schedule com duas cranes

PORT MANAGEMENT

Dashboard

FEATURES

Vessel

Dock

Storage Area

Visit Notification

Staff

Physical Resource

Qualification

Shipping Agent

Schedule

Daily Schedule

Default algorithm

Vessel Five

01/01/2026, 01:00:00 — 01/01/2026, 05:00:00

Crane: STS Crane 1
Staff: Staff One, Staff Two

Vessel Three

01/01/2026, 11:00:00 — 01/01/2026, 15:00:00

Crane: STS Crane 1
Staff: Staff One, Staff Two

Vessel Two

01/01/2026, 21:00:00 — 02/01/2026, 00:00:00

Crane: STS Crane 1
Staff: Staff One, Staff Two

Vessel Four

02/01/2026, 04:00:00 — 02/01/2026, 13:00:00

Crane: STS Crane 1
Staff: Staff One, Staff Two

Details

Running single-crane test...

Time to generate the shortest delay solution (secs): 0.00020813941955566406

Single-crane total delay: 43

Delays detected with single-crane, applying multi-crane...

Time to generate the shortest delay solution (secs): 0.0003578662872314453

Multi-crane total delay: 25

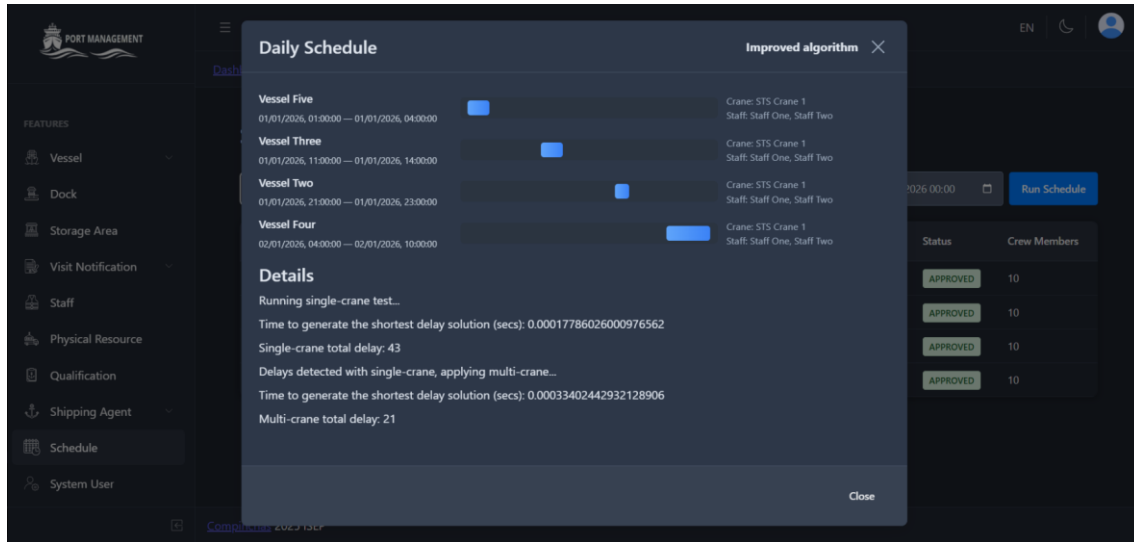
Close

01/01/2026 00:00

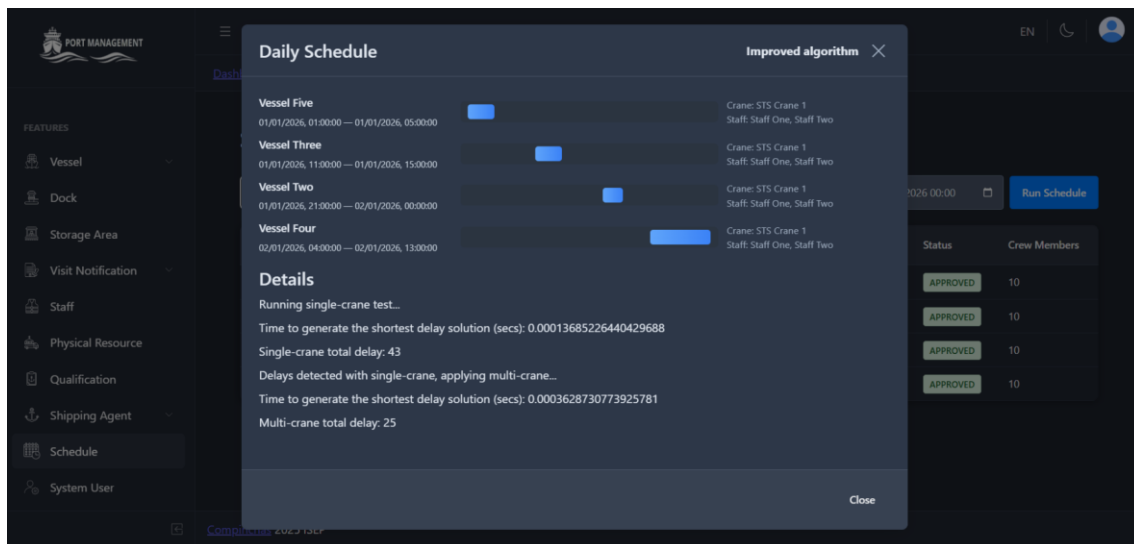
Run Schedule

Status	Crew Members
APPROVED	10
APPROVED	10
APPROVED	10
APPROVED	10

vessel_schedule com três cranes



Improved_vessel_schedule com duas cranes



Improved_vessel_schedule com três cranes

The screenshot displays the 'PORT MANAGEMENT' system interface. A 'Daily Schedule' window is open, showing a list of vessels and their scheduled times. An 'Improved algorithm' overlay is present, displaying details about the scheduling process.

PORT MANAGEMENT

FEATURES

- Vessel
- Dock
- Storage Area
- Visit Notification
- Staff
- Physical Resource
- Qualification
- Shipping Agent
- Schedule**
- System User

Daily Schedule

Improved algorithm ✕

Vessel	Time Range	Crane	Staff
Vessel Five	01/01/2026, 01:00:00 — 01/01/2026, 04:00:00	Crane: STS Crane 1	Staff: Staff One, Staff Two
Vessel Three	01/01/2026, 11:00:00 — 01/01/2026, 14:00:00	Crane: STS Crane 1	Staff: Staff One, Staff Two
Vessel Two	01/01/2026, 21:00:00 — 01/01/2026, 23:00:00	Crane: STS Crane 1	Staff: Staff One, Staff Two
Vessel Four	02/01/2026, 04:00:00 — 02/01/2026, 10:00:00	Crane: STS Crane 1	Staff: Staff One, Staff Two

Details

Running single-crane test...

Time to generate the shortest delay solution (secs): 0.00017786026000976562

Single-crane total delay: 43

Delays detected with single-crane, applying multi-crane...

Time to generate the shortest delay solution (secs): 0.00033402442932128906

Multi-crane total delay: 21

Run Schedule

Status	Crew Members
APPROVED	10
APPROVED	10
APPROVED	10
APPROVED	10

Close